

Dataset Upload

```
from google.colab import files

uploaded = files.upload()
```

Choose Files

Dataset.csv

Dataset .csv(text/csv) - 2249716 bytes, last modified: 8/6/2026 - 100% done
Saving Dataset .csv to Dataset .csv

Import Libraries

```
# Restaurant Data Analysis

## Section 1: Import Libraries
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Load Dataset

```
## Section 2: Load Dataset
```

```
import os

os.listdir()

['.config', 'Dataset .csv', 'sample_data']
```

```
df = pd.read_csv("Dataset .csv")

df.head()
```

	Restaurant ID	Restaurant Name	Country Code	City	Address	Locality	Locality Verbose	Longitude	Latitude	Cuisines	...	Current
0	6317637	Le Petit Souffle	162	Makati City	Third Floor, Century City Mall, Kalayaan Avenu...	Century City Mall, Poblacion, Makati City	Century City Mall, Poblacion, Makati City, Mak...	121.027535	14.565443	French, Japanese, Desserts	...	Botswai Pula(
1	6304287	Izakaya Kikufuji	162	Makati City	Little Tokyo, 2277 Chino Roces Avenue, Legaspi...	Little Tokyo, Legaspi Village, Makati City	Little Tokyo, Legaspi Village, Makati City, Ma...	121.014101	14.553708	Japanese	...	Botswai Pula(
2	6300002	Heat - Edsa Shangri-La	162	Mandaluyong City	Edsa Shangri-La, 1 Garden Way, Ortigas, Mandal...	Edsa Shangri-La, Ortigas, Mandaluyong City	Edsa Shangri-La, Ortigas, Mandaluyong City, Ma...	121.056831	14.581404	Seafood, Asian, Filipino, Indian	...	Botswai Pula(
3	6318506	Ooma	162	Mandaluyong City	Third Floor, Mega Fashion Hall, SM Megamall, O...	SM Megamall, Ortigas, Mandaluyong City	SM Megamall, Ortigas, Mandaluyong City, Mandal...	121.056475	14.585318	Japanese, Sushi	...	Botswai Pula(
					Third Floor,	SM	SM Megamall					

Data Understanding

```
df.shape
```

```
(9551, 21)
```

```
df.columns
```

```
Index(['Restaurant ID', 'Restaurant Name', 'Country Code', 'City', 'Address',
      'Locality', 'Locality Verbose', 'Longitude', 'Latitude', 'Cuisines',
      'Average Cost for two', 'Currency', 'Has Table booking',
      'Has Online delivery', 'Is delivering now', 'Switch to order menu',
      'Price range', 'Aggregate rating', 'Rating color', 'Rating text',
      'Votes'],
      dtype='object')
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9551 entries, 0 to 9550
Data columns (total 21 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Restaurant ID         9551 non-null   int64
 1   Restaurant Name       9551 non-null   object
 2   Country Code          9551 non-null   int64
 3   City                  9551 non-null   object
 4   Address               9551 non-null   object
 5   Locality              9551 non-null   object
 6   Locality Verbose      9551 non-null   object
 7   Longitude             9551 non-null   float64
 8   Latitude              9551 non-null   float64
 9   Cuisines              9542 non-null   object
10   Average Cost for two  9551 non-null   int64
11   Currency              9551 non-null   object
12   Has Table booking     9551 non-null   object
13   Has Online delivery   9551 non-null   object
14   Is delivering now     9551 non-null   object
15   Switch to order menu  9551 non-null   object
16   Price range          9551 non-null   int64
17   Aggregate rating      9551 non-null   float64
18   Rating color          9551 non-null   object
19   Rating text           9551 non-null   object
20   Votes                9551 non-null   int64
dtypes: float64(3), int64(5), object(13)
memory usage: 1.5+ MB
```

```
df.isnull().sum()
```

	0
Restaurant ID	0
Restaurant Name	0
Country Code	0
City	0
Address	0
Locality	0
Locality Verbose	0
Longitude	0
Latitude	0
Cuisines	9
Average Cost for two	0
Currency	0
Has Table booking	0
Has Online delivery	0
Is delivering now	0
Switch to order menu	0
Price range	0
Aggregate rating	0
Rating color	0
Rating text	0
Votes	0

dtype: int64

Data Cleaning

```
df = df.drop_duplicates()

df['Cuisines'] = df['Cuisines'].fillna('Unknown')

df.isnull().sum()
```

	0
Restaurant ID	0
Restaurant Name	0
Country Code	0
City	0
Address	0
Locality	0
Locality Verbose	0
Longitude	0
Latitude	0
Cuisines	0
Average Cost for two	0
Currency	0
Has Table booking	0
Has Online delivery	0
Is delivering now	0
Switch to order menu	0
Price range	0
Aggregate rating	0
Rating color	0
Rating text	0
Votes	0

dtype: int64

✓ Level 2 - Task 1: Restaurant Ratings Analysis

✓ Objective

Analyze the distribution of aggregate ratings and calculate the average number of votes received by restaurants.

```
rating_counts = df['Aggregate rating'].value_counts().sort_index()
rating_counts
```

	count
Aggregate rating	
0.0	2148
1.8	1
1.9	2
2.0	7
2.1	15
2.2	27
2.3	47
2.4	87
2.5	110
2.6	191
2.7	250
2.8	315
2.9	381
3.0	468
3.1	519
3.2	522
3.3	483
3.4	498
3.5	480
3.6	458
3.7	427
3.8	400
3.9	335
4.0	266
4.1	274
4.2	221
4.3	174
4.4	144
4.5	95
4.6	78
4.7	42
4.8	25
4.9	61

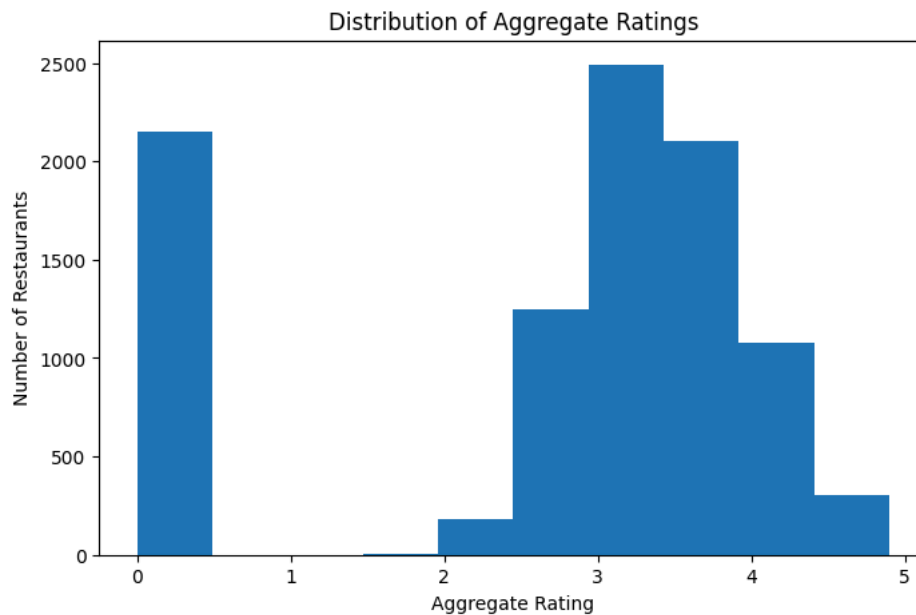
dtype: int64

```
plt.figure(figsize=(8,5))

plt.hist(df['Aggregate rating'], bins=10)

plt.title("Distribution of Aggregate Ratings")
plt.xlabel("Aggregate Rating")
plt.ylabel("Number of Restaurants")

plt.show()
```



```
average_votes = df['Votes'].mean()
```

```
average_votes
```

```
np.float64(156.909747670401)
```

Observation

- The distribution of aggregate ratings shows that most restaurants have ratings between 3.0 and 4.0.
- A large number of restaurants have a rating of 0.0, indicating that they are either not rated or have not received customer ratings.
- Very few restaurants have ratings below 2.0 or above 4.5.
- The average number of votes received by restaurants is approximately 156.91.

Conclusion

The analysis indicates that most restaurants have good customer ratings, generally between 3.0 and 4.0. Although many restaurants have not received ratings (0.0), the average number of votes (156.91) suggests that customers actively rate and review restaurants in the dataset.

✓ Level 2 - Task 2: Cuisine Combination Analysis

✓ Objective

Identify the most common cuisine combinations and analyze whether certain cuisine combinations have higher average ratings.

```
cuisine_combination = df['Cuisines'].value_counts()
```

```
cuisine_combination.head(10)
```

	count
Cuisines	
North Indian	936
North Indian, Chinese	511
Fast Food	354
Chinese	354
North Indian, Mughlai	334
Cafe	299
Bakery	218
North Indian, Mughlai, Chinese	197
Bakery, Desserts	170
Street Food	149

dtype: int64

```
cuisine_rating = df.groupby('Cuisines')['Aggregate rating'].mean()
cuisine_rating.sort_values(ascending=False).head(10)
```

	Aggregate rating
Cuisines	
Italian, Deli	4.9
American, Coffee and Tea	4.9
American, BBQ, Sandwich	4.9
BBQ, Breakfast, Southern	4.9
Italian, Bakery, Continental	4.9
American, Burger, Grill	4.9
Hawaiian, Seafood	4.9
Mexican, American, Healthy Food	4.9
American, Caribbean, Seafood	4.9
Mughlai, Lucknowi	4.9

dtype: float64

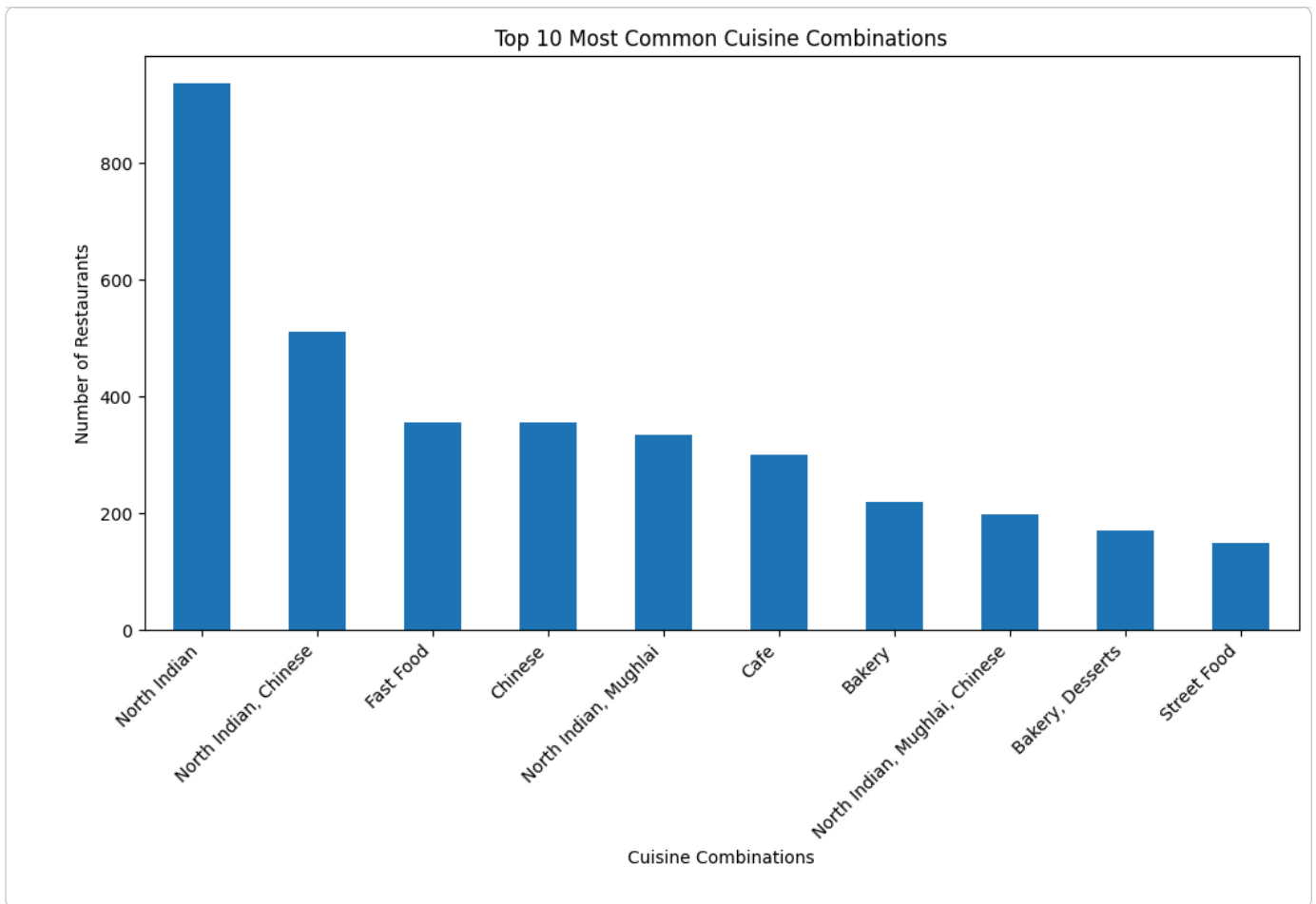
```
plt.figure(figsize=(12,6))

cuisine_combination.head(10).plot(kind='bar')

plt.title("Top 10 Most Common Cuisine Combinations")
plt.xlabel("Cuisine Combinations")
plt.ylabel("Number of Restaurants")

plt.xticks(rotation=45, ha='right')

plt.show()
```



Observation

- North Indian is the most common cuisine combination with 936 restaurants.
- North Indian, Chinese is the second most common cuisine combination with 511 restaurants.
- Fast Food and Chinese cuisines are also popular among restaurants.
- Some cuisine combinations such as Italian, Deli and American, Coffee and Tea have the highest average rating of 4.9.
- Popular cuisine combinations do not always have the highest ratings.

Conclusion

The analysis shows that North Indian and North Indian, Chinese are the most common cuisine combinations in the dataset. However, the highest-rated cuisine combinations are different, with several combinations achieving an average rating of 4.9. This indicates that popularity and customer ratings are not always directly related.

✓ Level 2 - Task 3: Geographic Analysis

✓ Objective

Analyze the geographical distribution of restaurants using latitude and longitude coordinates and identify clusters of restaurants.

```
!pip install folium
```

```
Requirement already satisfied: folium in /usr/local/lib/python3.12/dist-packages (0.20.0)
Requirement already satisfied: branca>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from folium) (0.8.2)
Requirement already satisfied: Jinja2>=2.9 in /usr/local/lib/python3.12/dist-packages (from folium) (3.1.6)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from folium) (2.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from folium) (2.32.4)
Requirement already satisfied: xyzservices in /usr/local/lib/python3.12/dist-packages (from folium) (2026.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2>=2.9->folium) (3.0.3)
```

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (3.18)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->folium) (2026.6.17)

```
import folium
```

```
restaurant_map = folium.Map(  
    location=[df['Latitude'].mean(), df['Longitude'].mean()],  
    zoom_start=2  
)  
  
restaurant_map
```



 Leaflet | © OpenStreetMap contributors

```
restaurant_map = folium.Map(  
    location=[df['Latitude'].mean(), df['Longitude'].mean()],  
    zoom_start=2  
)  
  
for i in range(100):
```

```
folium.Marker(  
    location=[df.iloc[i]['Latitude'], df.iloc[i]['Longitude']],  
    popup=df.iloc[i]['Restaurant Name']  
).add_to(restaurant_map)  
  
restaurant_map
```



Leaflet | © OpenStreetMap contributors

Observation

- The map displays restaurant locations using latitude and longitude coordinates.
- Restaurants are spread across different countries and cities.
- Multiple restaurant locations are concentrated in specific regions, forming visible clusters.
- The visualization indicates that restaurants are more densely located in urban areas.

Conclusion

The geographic analysis shows that restaurants are not evenly distributed. Most restaurants are concentrated in major cities and urban regions, creating clusters. Mapping restaurant locations helps identify areas with high restaurant density.

- Level 2 - Task 4: Restaurant Chains Analysis
- Objective

Identify restaurant chains in the dataset and analyze their ratings and popularity.

```
restaurant_chains = df['Restaurant Name'].value_counts()
restaurant_chains.head(10)
```

Restaurant Name	count
Cafe Coffee Day	83
Domino's Pizza	79
Subway	63
Green Chick Chop	51
McDonald's	48
Keventers	34
Pizza Hut	30
Giani	29
Baskin Robbins	28
Barbeque Nation	26

dtype: int64

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.